Proposal for the Design of FUZE: A System for Whole-Board Constraint Validation of Electronic Bills of Materials

Jerral Chua Fung Yong
Kataruka Mohika
Jethani Samarth
Sinha Armaan
Jain Sparsh
Faye Chan
Lai Yi

Team FUZE
College of Computing and Data Science, Nanyang Technological University

Submitted to—

Ajith Santhisenan

College of Computing and Data Science, Nanyang Technological University

Table of Contents

Executive Summary 3
Statement of Problem 4
Background 4
Previous Related Work 5
Patent Search 6
Objectives 6
Technical Approach 7
- Needs of Customers 8
- Target Specifications 8
- Technology Consideration 12
- System Architecture/Platform 14

Project Management 16
- Deliverables 20
- Budget 22
- Communication and Coordination with Sponsor 24
- Team Qualifications 26

Conclusion 27
References 28
Appendix A: Résumés of Team Members 29
- Mohika Kataruka 30
- Faye Chan 31
- Samarth Jethani 32
- Armaan Sinha 33
- Jerral Chua Fung Yong 34
- Sparsh Jain 35
- Lai Yi 36

Executive Summary

Hardware engineers spend a large share of their design time confirming that independently chosen components will work together. A survey of 439 engineers found that 77 percent spend five hours or more each week on this work [Accuris and Fuld, 2026]. The average printed circuit board still requires 2.9 respins, at approximately $28,500 each [Siemens EDA, YEAR].

No current tool performs the check. Distributor search locates individual parts, and computer-aided design tools capture and verify a schematic. Neither evaluates whether a chosen set of parts is mutually consistent. Compatibility is a property of the whole board, and every available interface takes a single part as its unit of query.

This document proposes the design of FUZE, a web application that converts a plain-language description of a circuit board into a bill of materials validated as a graph. The system divides the work between two components. A deterministic constraint engine determines what is broken, a language model selects a resolution strategy, and the engine then re-validates the result. Ten rules covering voltage, current, thermal dissipation, interface roles, pin count, availability, temperature grade, footprint, energy budget and rail coverage will be implemented as pure functions with no network access and no model in their path. The same board therefore produces the same verdicts on every run. Each verdict names the field that decided it, quotes that field's value, and identifies the source it was read from. No compatibility judgement is ever produced by a language model.

A seven-member team will deliver the system in six phases across fourteen weeks, working behind an event contract frozen in Week 5 so that the engine, the interface and the test suites can proceed in parallel. The requested budget is $22,517.20, of which 98 percent is personnel.

Statement of Problem

A printed circuit board carries a set of components all connected by copper traces. Before any of that is drawn, someone must decide which components to use. That decision produces the bill of materials, a list of manufacturer part numbers with quantities and the role each part plays. Everything downstream depends on it. Procurement buys from it, assembly builds from it, and an error in it survives every check that follows.

A bill of materials can be internally inconsistent in ways that no drawing tool detects. A 3.3-volt sensor placed on a 5-volt rail is destroyed on first power-up, and the schematic tool will draw the connection without objection. A regulator rated for 600 mA supplying a radio that draws 500 mA in transmit and a display that draws 200 mA will brown out under load, though only intermittently, which is the most difficult class of fault to

diagnose. A linear regulator converting 5 V to 3.3 V at 400 mA dissipates 0.68 W as heat; in a SOT-23-5 package with a thermal resistance near 200 °C/W, that produces a junction temperature rise of roughly 136 °C, well beyond the part's rating. A part may also be correct in every electrical respect while the distributor holds none of it at the quantity a production run requires.

None of this arithmetic is difficult. It is distributed across forty datasheets, and no tool performs it for the board as a whole.

Existing tools each address one part of the problem. KiCad and Altium verify that the drawing is well-formed, checking that no pin is left floating and that footprints match, through a design rule check. They do not read datasheets. Octopart and DigiKey locate parts and report price and stock, but they have no knowledge of what else is on the board. The gap between the two exists because compatibility is a whole-board property, while every available interface takes a single part as its unit of query.

Background

The sponsor has requested a system that reduces the hours spent selecting and cross-checking components, and reduces the respins those selections cause. Stated in the sponsor's own terms, the requirement is a tool that answers one question: given this set of parts and this application, does the board work, and if not, which number is wrong and where did it come from?

Three conditions have been specified. Judgements must be checkable, so that an engineer can trace any verdict to a named field in a named source. The system must address supply as well as physics, because a part that is electrically correct and unavailable is not a solution. It must also be usable by a non-expert, since much of the wasted effort falls on junior engineers and student teams who do not yet know which cross-checks matter.

What this problem costs has been measured. A survey of 439 engineers across aerospace and defence, electronics, automotive, medical devices and industrial manufacturing found that 77 percent spend five hours or more every week reading datasheets and comparing component alternatives [Accuris and Fuld, 2026]. The same survey found that 58 percent spend more than thirty hours a month extracting datasheet data by hand. A separate study covering 128,000 engineers reports that 48 percent spend an hour or more each day searching for parts [CADENAS, 2022]. Five hours a week is roughly an eighth of an engineer's working time, spent on a task that is arithmetic rather than design.

That effort is repeated. Sixty-eight percent of design projects make six or more component changes, and 73 percent of boards lose at least one component to a supply, lifecycle or compliance issue [Accuris and Fuld, 2026; Lifecycle Insights, 2022]. Each change invalidates the cross-checks already performed on the parts around it.

The cost rises sharply when a problem is found late. Of those surveyed, 51 percent report that more than 11 percent of their designs require component changes after design freeze, and 46 percent estimate that a single such change costs over $50,000 [Accuris and Fuld, 2026]. The average printed circuit board requires 2.9 respins before release, each costing approximately $28,500 and sixteen days [Siemens EDA, YEAR]. The problem is

therefore inexpensive to prevent and expensive to discover, which is the case for automating the checks.

The literature is not fully consistent on these figures. Altium's whitepaper puts a respin at $46,000 against Lifecycle Insights' $28,500 [Altium, 2026]. This proposal uses $28,500 throughout due to stronger source.

Previous Related Work

Four bodies of work are adjacent to this project, and the boundary of the proposed scope is defined by where each of them stops.

The first is component search. Octopart, DigiKey, Mouser and JLCPCB's parts library expose distributor inventory as structured, queryable data. This includes price breaks, stock levels and, in some cases, lifecycle status. Their limitation is architectural: the unit of query is a single part, and no interface exists for asking a question about a set of parts.

The second is electronic design automation. KiCad and Altium Designer are the established tools for schematic capture and layout. Both perform electrical rule checking. Those checks read the drawn netlist, covering connectivity, pin types and unconnected nets instead of datasheet parameters. Altium's ActiveBOM and Siemens' supply-chain products add lifecycle and stock awareness to the bill of materials, but neither validates parts against each other electrically.

The third is automated board synthesis. We are excluding this from the project. atopile describes hardware as code, Quilter automates layout, JITX generates circuits programmatically, and Flux.ai assists schematic capture. Work in this area has repeatedly arrived at a finding that the team's own interviews reproduced: engineers want compatibility checking they can audit rather than a generated board they cannot. Synthesis also requires pin-level connectivity, a substantially larger data problem than block-level validation. This cannot be delivered responsibly within fourteen weeks.

The fourth is the application of language models to engineering documents. A large portion of recent literature covers retrieval over technical PDFs. Most general-purpose assistants can state whether a sensor is compatible with a microcontroller. The failure mode here is that models produce fluent, confident, unsourced claims, and an incorrect compatibility claim is indistinguishable in tone from a correct one. The response adopted in this proposal is to assign the model the judgement work it performs well while leaving it structurally unable to produce a number.

The team will also conduct its own interviews. A preliminary round of fifteen, across university electrical engineering programmes, a makerspace, an IEEE student branch, and competitive robotics and rocketry teams, changed the framing of this proposal [Team FUZE, 2026]. The interviewed engineers described difficulty arises after a chosen part proves unavailable and a replacement must be found that still fits everything already committed.

Patent Search

A freedom-to-operate review will be completed in Week 2 and appended to this proposal. The search will cover USPTO full-text, Google Patents, Espacenet and the register of the Intellectual Property Office of Singapore, across the Cooperative Patent Classification classes G06F 30/30–30/39 (computer-aided design of electronic circuits), G06F 30/398 (design verification and rule checking), G06Q 10/087 (inventory and supply-chain management) and G06Q 30/0601 (electronic catalogue and product-selection systems). Search terms will include bill of materials validation, component compatibility checking, automated component selection, and supply chain aware design.

The team's preliminary assessment is that the contribution is a system architecture rather than an algorithm. Specifically, it is the division of labour between a deterministic engine and a fenced language model. The constraint checks themselves are textbook electrical engineering and are not patentable subject matter. No patent numbers are asserted in this proposal. Anything the Week 2 search identifies as material will be reported to the sponsor in writing before implementation begins on the affected component.

Objectives

This document proposes the design of FUZE, a system that converts a plain-language description of a circuit board into a bill of materials validated as a graph and traceable to its sources. The project has four objectives:

(1) a deterministic constraint engine that evaluates every rule against every part after every placement and cites its evidence;

(2) a sourcing pipeline that fills each component slot with a real, orderable part and preserves the origin of every value it reads;

(3) a fenced repair loop in which a language model selects a strategy from a set the engine has computed, and the engine re-validates the result; and

(4) a streaming web application that allows an engineer who did not produce a judgement to check it.

The engine is the technical core. It will contain ten rules covering voltage overlap, current budget, thermal dissipation, interface role matching, pin budget, availability, temperature rating, footprint, energy budget and rail coverage. Each is a pure function from a board to a list of verdicts. Each verdict names the field that decided it, quotes that field's value, and identifies where the value came from. A rule that cannot evaluate returns unchecked and names the missing field, and it never substitutes a default. Because the engine holds no network client, no language model and no read of the system clock, the same inputs produce the same verdicts on every run.

The second objective exists because a validated board made of parts with no stock is incorrect. The pipeline will push numeric constraints into the distributor query rather than filtering after retrieval, then normalise free-text catalogue values into a single typed specification.

The third objective addresses what the interviews identified as a bottleneck. Resolving a conflict without breaking what is already committed is the difficult part. On a failure the engine computes a legal set, the participants that may be changed are ranked so that the cheapest change comes first. The model then selects one strategy from that set. The selection is validated against the set before it is applied, and an illegal or timed-out choice falls back to the least disruptive candidate. Repairs per slot are capped at three, after which the conflict escalates to the user as an explicit trade-off.

The fourth objective exists to make sure that the judgements are checkable. An engineer must be able to disagree with the tool and establish who is right in under a minute without leaving the interface. In practice this means a conflict displays every rule that ran on the failing part, including those that passed, rather than only the rule that failed.

What the project will not do matters as much. FUZE will not generate a schematic. It will not lay out or route a board, assign pins, produce a netlist, simulate a circuit, or emit manufacturing files. The boundary is drawn at block-level validation as prior work, prelim interviews and practical data limitations make board synthesis impractical.

Technical Approach

The proposed approach is based on the following design: the engine decides what is broken and the model decides what to do about it. After the model makes its decision, the engine re-checks the model's work.

Deterministic code is well-suited to arithmetic but bad at judgement, and the opposite holds true for a language model. The architecture thus gives each side the half it is good at and makes them meet over a vocabulary that is defined by the engine.

This architectural separation provides three practical advantages.

- Every verdict is reproducible because the language model never produces any pass or fail. This allows the engineers to verify the result consistently during design review without having to independently re-check the model's compatibility judgements.
- An incorrect model decision is not the direct cause of an invalid board passing validation. The model is only responsible for selecting the repair strategy. After this is done, the constraint engine re-runs all the applicable rules. Therefore, the model is not allowed to declare that two components are compatible by this scheme.
- This separation reduces the impact of prompt-injection attempts on the validation results. If a user brief contains instructions asking the system to report that all checks have passed, the language model has no mechanism through which it can produce a pass verdict. Validation remains controlled by the deterministic engine rather than by prompt-level filtering.

Accounting for revalidation also provides an additional advantage. Every constraint rule is re-evaluated after a component is added or replaced, which results in even previously accepted components being checked again when the board configuration changes. This is an important feature since component compatibility depends on the entire board rather

than on individual parts in isolation. Electrical conflicts and sourcing issues therefore pass through the same detect, rank, repair and revalidate loop.

The following subsections describe the customer needs that are addressed by this approach, the target specifications, selected technologies, and system architecture.

Customer Needs

The customer needs listed below were derived from the sponsor's request and were further refined through the fifteen preliminary interviews described in the problem statement. Six primary customer needs were identified:

- Reduced manual cross-checking effort. The system should reduce the time engineers spend manually comparing component specifications by automatically performing relevant checks across the entire board whenever a component is added or changed.
- Auditable validation results. Produced judgements should be traceable to the underlying data and source so that an engineer can quickly and easily verify the result within the interface.
- Supply-chain awareness. Since sourcing issues can require component changes and redesign, factors such as availability, lifecycle status and lead time should be treated as design constraints alongside electrical constraints such as current and voltage.
- Explicit handling of missing information. When required data is unavailable, the system should report the corresponding check as unchecked rather than assuming a value or producing a false pass.
- Orderable output. The final bill of materials should map to real manufacturer part numbers with current pricing and stock information.
- Accessibility to non-expert users. The system should be usable by junior engineers and students without requiring them to know in advance which compatibility checks need to be performed.

Target Specifications

Table 1 lists the specifications that are used to evaluate the final system. Each specification has a clear acceptance criterion so that it can be tested objectively.

ID	Specification	Acceptance criterion
DS-1	Rule coverage	The engine shall implement at least 10 constraint rules, and each rule must run without network calls, model calls, or access to the system clock.
DS-2	Determinism	The same brief evaluated against the same cached part data shall produce a byte-identical verdict set

		across 100 consecutive runs.
DS-3	Origin	100% of emitted verdicts shall carry field name, verbatim field value, and source identifier, either as a resolvable URL or a named reference table. Zero verdicts with a null source.
DS-4	No substituted defaults	If required data is missing, the rule shall return unchecked and identify which field is missing. No default values may be substituted. Tested across at least 40 boards.
DS-5	End-to-end latency	A validated BOM shall be produced within 130 seconds using live distributor data, and within 35 seconds using recorded data, for boards with up to 12 component slots.
DS-6	Streaming responsiveness	The first event shall appear within 2 seconds of submission, and event sequences must contain no gaps or duplicates, including after pause and resume.
DS-7	Bounded repair	No component slot shall be repaired more than 3 times before escalating to the user, and no run shall fail to terminate. Verified against 10 unsatisfiable boards.
DS-8	Test coverage	Backend test coverage shall be at least 80%, with at least 95% coverage for the constraint engine.
DS-9	Sourceability	Every BOM item shall map to a real manufacturer part number with sufficient distributor stock, or be clearly flagged if the requirement cannot be met.
DS-10	Model fencing	Every repair proposed by the language model shall be checked against the engine's allowed options before it is applied. No unvalidated repair may be accepted.
DS-11	Export fidelity	The exported CSV shall show the final post-repair board and include part number, quantity, price, stock and datasheet URL. It must open correctly in Excel and Google Sheets.
DS-12	Availability	The deployed system shall maintain at least 99% availability during the two-week demonstration

		period. Cold start must be within 60 seconds and warm health responses within 500 ms.
DS-13	Security	Passwords shall use argon2id, session cookies shall be secure, and no secrets may be committed to version control.
DS-14	Injection resistance	At least 25 prompt-injection test briefs shall produce zero false pass verdicts.
DS-15	Usability	At least 8 out of 10 first-time users shall be able to run a design and identify the cause of a conflict without assistance within 10 minutes.

Table 1: Target specifications and their acceptance criteria

The ten rules that are referred to by DS-1 are set out in Table 2.

Rule	Detects	Core relation
voltage_overlap	A component receiving a voltage outside its supported range	FAIL unless part.vmin ≤ rail.v ≤ part.vmax
current_budget	A power rail requiring more current than its source can provide	FAIL if Σ i_peak × (1 + margin) > source.i_max, where margin = 0.15
thermal_dissipation	A regulator exceeding its safe operating temperature	Linear: P = (vin − vout) × I; Switching: P = vout × I × (1/η − 1); ΔT = P × θJA; FAIL if ambient + ΔT > part.temp_max
interface_role_match	A peripheral requiring an interface that no controller supports	FAIL unless a compatible interface exists between the peripheral and controller
pin_budget	More pins being required than the microcontroller provides	FAIL if Σ peripheral.pins_required > mcu.pins_available

availability	Insufficient stock or an unsuitable lifecycle status	FAIL if stock < req.min_stock; WARN if lifecycle is NRND or obsolete
temperature_rating	A component that cannot operate within the required temperature range	FAIL unless part.temp_min ≤ req.range[0] and part.temp_max ≥ req.range[1]
footprint	A package unsuitable for the intended assembly process	WARN only
energy_budget	A power source that cannot support the required operating time	FAIL if capacity_mAh / continuous_draw_mA < required_runtime_h
rail_coverage	A component that is not connected to any power rail	Reported as unchecked, never pass

Table 2: Constraint rule set

For these specifications to be met, there are four critical design issues that need to be addressed.

1. Origin must be preserved throughout the architecture from distributor payload through normalisation and into the engine. Therefore, this must be included in the part specification type before the rules are implemented.
2. Determinism must be maintained despite language-model-based normalisation: Parsing free-text catalogue values may require a language model, while DS-2 requires deterministic results.
3. The interface contract is a critical dependency as seven team members cannot develop different parts without having agreed on an event schema. This interface must be defined and finalised early enough to allow for parallel development.
4. Missing data must be handled explicitly by the engine rather than asserting an unsupported point value.

In addition to these internal design considerations, the system must also operate within several external technical and resource constraints.

Constraint	Consequence for the design
Distributor APIs have rate limits	Search queries must be filtered efficiently, and recorded data will be used for repeatable testing.

Nexar has a very limited free tier	Nexar will not be used as a primary data source.
Some thermal values are not provided for individual parts	The system will use recognised reference data where appropriate and clearly identify the source used.
Duty-cycle information is often unavailable	Thermal checks will use conservative assumptions and clearly state them in the result.
Server-sent event streams have scaling limitations	The initial system will use a single application worker; horizontal scaling is outside the current scope.
Free-tier hosting may enter sleep mode	Cold-start delays must be considered and mitigated during the demonstration period.
Language-model API access may depend on region	The model endpoint and API configuration must use compatible regions.

Table 3: Constraints on the design and their consequences

These constraints also define the boundaries of the delivered system. FUZE will validate a board at the block level only, and its results will depend on the quality and completeness of the parameters that are available to the system. A component with a sparse catalogue may hence accumulate unchecked verdicts. While this is the intended behaviour, an unchecked result should never be interpreted as fully validated.

Technology Consideration

Table 4 lists the main technologies that are proposed for each layer of FUZE. The choices are guided by two requirements:

1. The constraint engine must be able to operate independently of the language model and external service.
2. The data layer must be able to preserve the source of every component value used during validation.

Layer	Selection	Rationale
Frontend	React 19, TypeScript, Vite, Tailwind CSS	Provides a modern interface stack and supports strongly typed communication with the backend.

Interface layer	FastAPI, Python 3.11+	Supports asynchronous APIs and server-sent events for streaming system progress to the frontend.
Orchestration	LangGraph	Provides streaming, pause and resume functionality, and persistent run state required by the workflow.
Constraint engine	Pure Python	Keeps validation deterministic, easy to test, and independent of external services and the language model.
Component parameters, stock and price	JLCPCB	Provides component specifications, availability and pricing that can be used when generating an orderable BOM.
Lifecycle and datasheet information	Mouser	Supplements JLCPCB with lifecycle information and datasheet links that may not otherwise be available.
Datasheet fallback	Web search and manufacturer datasheets	Provides additional component information when catalogue sources are incomplete.
Language model	GLM-5.2	Used for interpreting user requirements, normalising catalogue data and selecting repair strategies.
Persistence	Managed PostgreSQL	Stores user accounts, projects, runs and workflow state.
Tool integration	Model Context Protocol (MCP)	Provides a common interface for connecting external data sources and allows them to be replaced more easily.
Testing	pytest, pytest-cov, Playwright	Supports unit, integration, coverage and end-to-end browser testing.
Deployment	Render and manage PostgreSQL	Provides a low-cost deployment environment suitable for the project demonstration.

Table 4: Proposed technologies and their rationale

From the table, there are two technology choices that require further explanation.

1. LangGraph is being used to manage the workflow. This is done as it allows to directly stream intermediate results to the user, pause and resume a workflow when clarification is required and store the state of each run. These capabilities reduce the amount of custom workflow-management logic that the team needs to implement.
2. Since no single source provides all the information which is required by the engine, the system uses multiple component data sources like JLCPCB which provides component parameters like stock and pricing, and Mouser which provides lifecycle information and datasheet links where available. Manufacturer datasheets will be used when additional technical specifications are required.

System Architecture/Platform

Figure 1 shows the overall architecture of our project FUZE. The key design feature is the separation of the constraint engine from the language model and external services. The constraint engine performs all validation independently and hence helps ensure that its results are deterministic and reproducible.

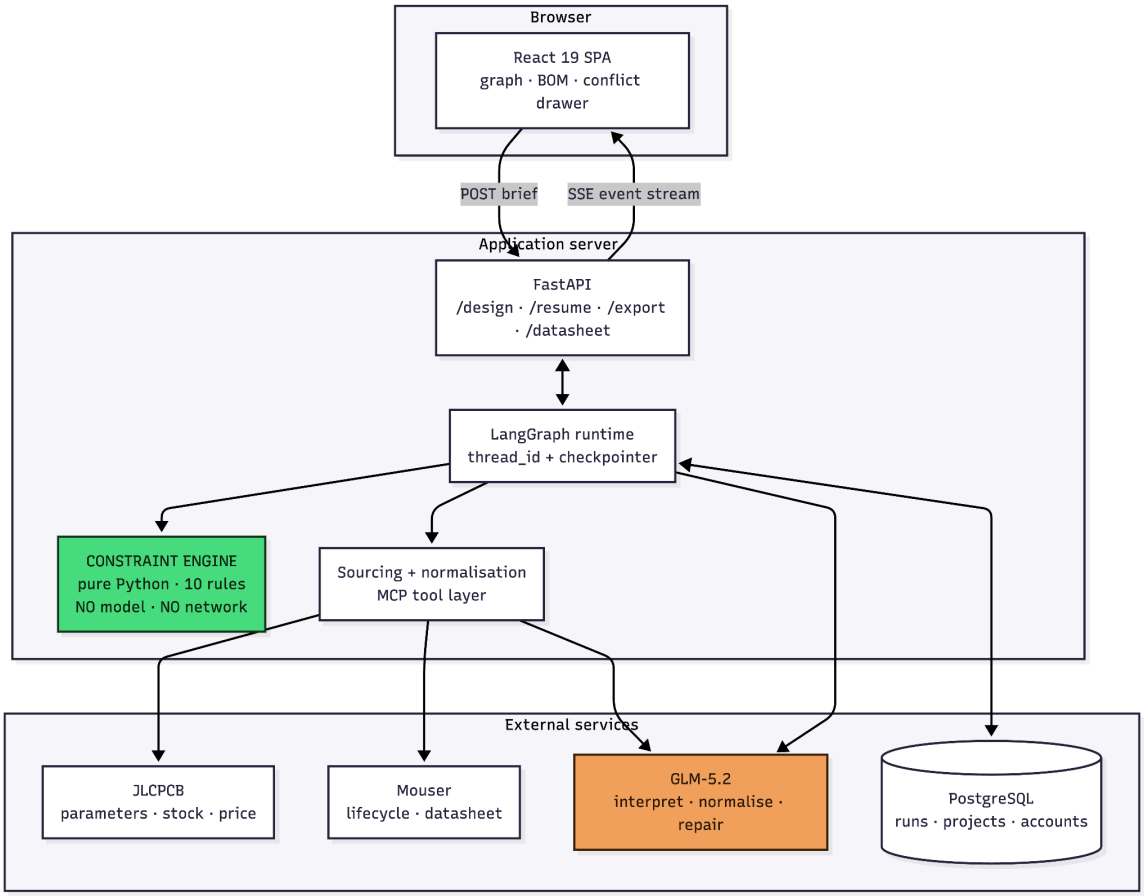


Figure 1: Overall FUZE system architecture, showing the deterministic constraint engine separated from model and external-service components.

Figure 2 shows how validation and repair are handled. The constraint engine first checks the board for conflicts. If a conflict is found, it generates a set of valid repair options for the language model to choose from. The selected repair is then applied and the board is validated again. If no valid repair is available, or the repair limit is reached, the issue is escalated to the user.

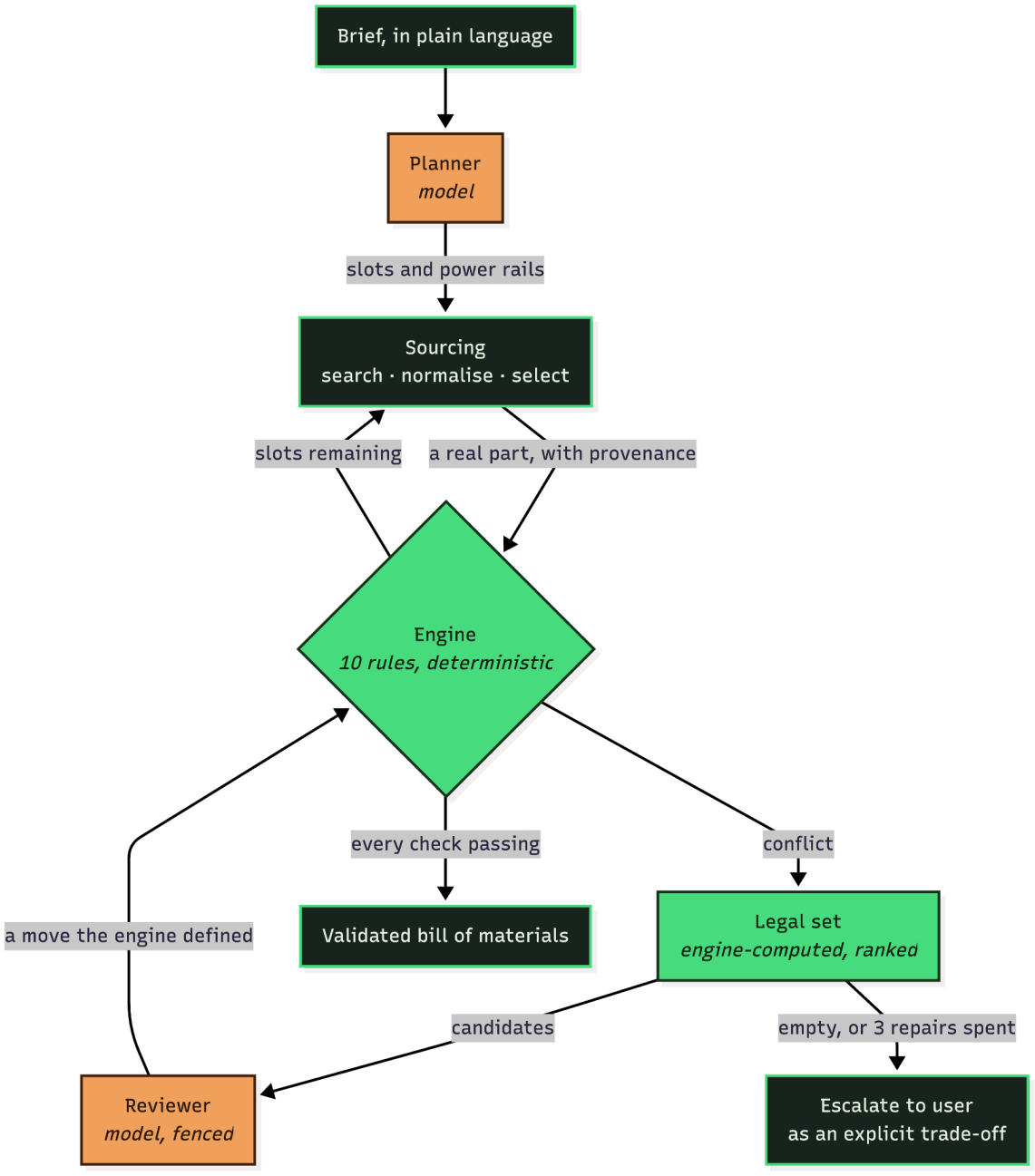


Figure 2: Validation and bounded repair loop using engine-computed legal repair actions.

Figure 3 shows the same workflow in greater detail. The system first interprets the user's requirements and asks for clarification when necessary. It then searches for components, normalises their data, selects suitable parts, and validates the board. Any conflicts enter the repair process before validation is repeated. Once all required components pass validation, the final bill of materials is produced.

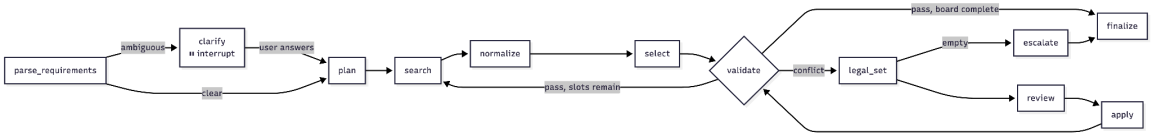


Figure 3: Detailed LangGraph execution flow from requirement parsing to validated BOM generation.

The system will be organised into separate backend and frontend modules. The backend will include packages for the constraint engine, component sourcing and normalisation, requirement planning, workflow orchestration, and the API. The frontend will include the component graph, bill of materials, conflict information, project screens, and streaming functionality.

Development will follow the dependency structure of the system. Unit, integration, and end-to-end tests will be used throughout development to verify both individual components and the complete system.

Project Management

The project will run for fourteen weeks in six phases, beginning Thursday 20 August 2026, and completing Wednesday 25 November 2026.

Figure 4 presents the schedule as a Gantt chart.

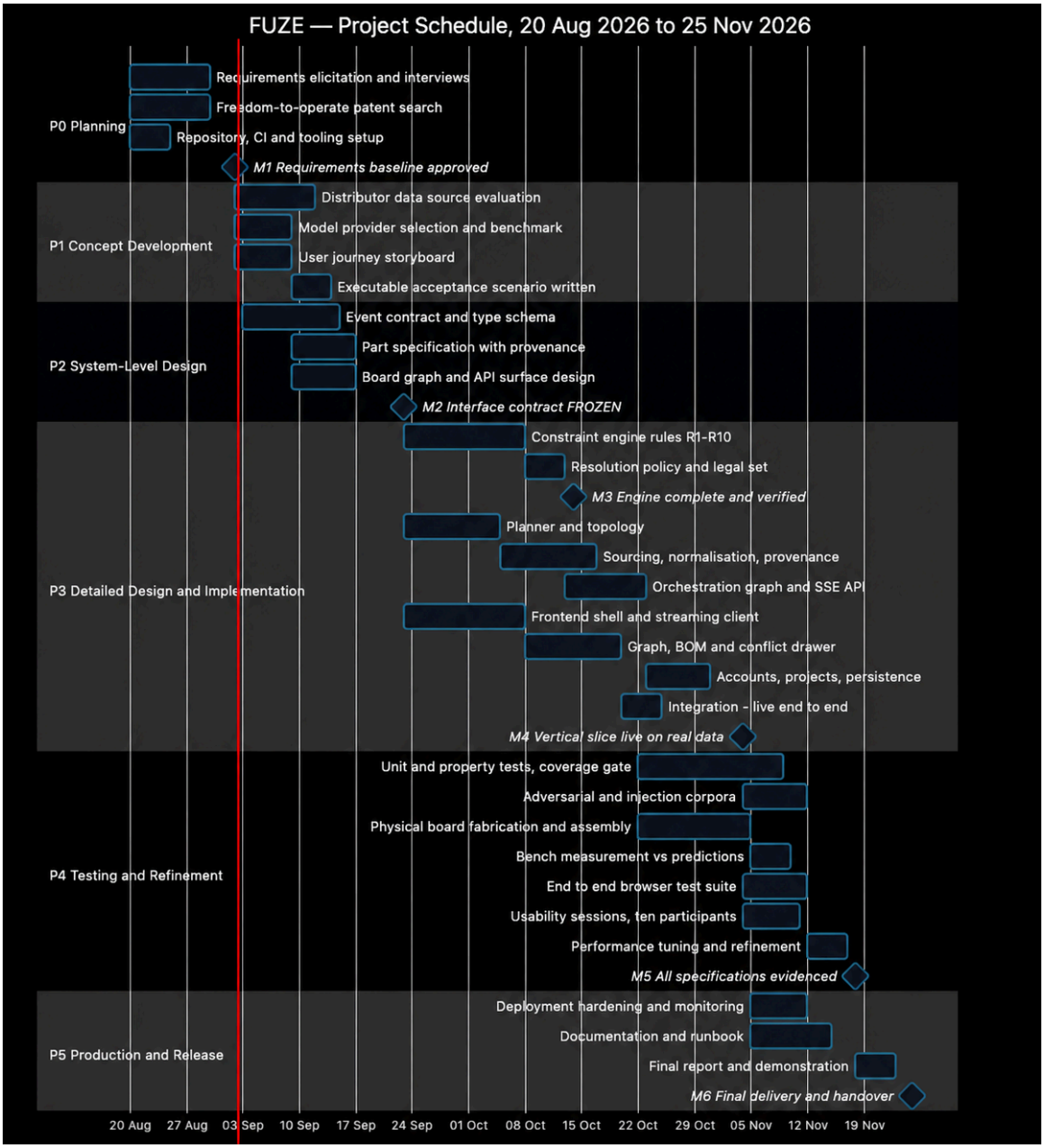


Figure 4: Gantt chart for the project

Table 5 gives each phase its objective and its exit criterion. A phase remains open until that exit criterion has been demonstrated to the sponsor at the milestone review. Table 6 lists the milestone dates and their dependencies.

Phase	**Weeks**	**Objective**	**Exit criterion**
P0 – Planning	1–2	Requirements elicitation, stakeholder interviews,	Requirements baseline set (M1)

		freedom-to-operate search, tooling and repository setup	
P1 – Concept Development	2–4	Evaluate distributor sources against the rule set's actual field needs; select the model provider; storyboard the primary user journey; write the acceptance scenario	Data-source selection memo accepted; one executable acceptance scenario written and failing
P2 – System-Level Design	3–5	Define the event contract, the part specification type with its origin, the board graph model and the interface surface	Interface contract frozen (M2)
P3 – Detailed Design and Implementation	5–11	Build the engine, planner, sourcing pipeline, repair loop, interface layer and frontend, bottom-up	End-to-end vertical slice running against live distributor data (M4)
P4 – Testing and Refinement	10–13	Unit, integration, adversarial and end-to-end testing; usability testing; performance tuning	Every specification in Table 1 met and evidenced (M5)
P5 – Production and Release	12–14	Deployment hardening, documentation, runbook, final report and demonstration	Final delivery (M6)

Table 5: Task phases, objectives and exit criteria

ID	Milestone	Target date	Week	Depends on
M1	Requirements baseline approved	Wed 2 Sep 2026	2	—
M2	Interface contract frozen	Wed 23 Sep 2026	5	M1
M3	Constraint engine complete and	Wed 14 Oct	8	M2

	verified	2026		
M4	End-to-end vertical slice live on real data	Wed 4 Nov 2026	11	M3
M5	Validation complete; all specifications evidenced	Wed 18 Nov 2026	13	M4
M6	Final delivery, demonstration and handover	Wed 25 Nov 2026	14	M5

Table 6: Milestone schedule and dependencies

Table 7 divides the work among team members. Each member is named against specific milestones and specific target specifications.

Member	**Role**	**Primary responsibilities**	**Accountable for**
Mohika	Project Manager	Schedule ownership, sponsor liaison, weekly status reporting, change control, risk register, meeting minutes, budget tracking	M1, M6; all sponsor communication
Sparsh	Lead Developer	Technical direction and final design authority; the interface contract; the engine's rule set and resolution policy; architecture review of all merged work	M2, M3; DS-1, DS-2, DS-3, DS-4, DS-7, DS-10
Armaan	Backend Developer	Interface layer, orchestration graph, sourcing and normalisation pipeline, distributor adapters, datasheet ingestion, persistence and authentication	M4; DS-5, DS-6, DS-9, DS-13
Lai Yi	Frontend Developer	React application, streaming client, component graph, conflict drawer, bill-of-materials table, accounts and project screens, accessibility	M4; DS-11, DS-15, and the client half of DS-6

Samarth	QA Manager — Verification	Unit and property-based tests for every rule; the regression board corpus; coverage gates; hand-calculation cross-checks of each rule's arithmetic; adversarial and injection corpora	DS-1, DS-4, DS-8, DS-14
Faye	QA Manager — Validation	End-to-end browser tests; acceptance scenarios; usability sessions with external participants; the sponsor acceptance evidence pack	M5; DS-14, and the Table 1 evidence pack
Jerral	Release Engineer	Continuous integration and deployment, environment and secret management, deployment blueprint, database migrations, monitoring, fixture recording, release tagging and the runbook	DS-12, and the infrastructure half of DS-13

Table 7: Division of responsibilities among team members

The team will work trunk-based, on short-lived branches. Every change requires one review approval from an engineer other than its author. The Lead Developer reviews any change touching the engine or the contract.

Deliverables

The team will provide twelve deliverables, listed in Table 8 with the date each is due. Every item is delivered to the sponsor's nominated repository or shared drive on or before that date, and the Project Manager confirms receipt in writing. Deliverables cluster on the milestone dates by design, so that every phase gate is accompanied by an artefact the sponsor can inspect rather than a verbal report of progress.

#	**Deliverable**	**Description**	**Due**
D1	Requirements specification	Functional and non-functional requirements traced to the specifications in Table 1; interview findings; freedom-to-operate memo	Wed 2 Sep 2026 (M1)

D2	Interface contract	Frozen event schema, type definitions, interface surface and ordering guarantees, as machine-readable JSON Schema plus prose	Wed 23 Sep 2026 (M2)
D3	System design document	Architecture, module decomposition, data-flow diagrams, the ten rules with their formulas and stated assumptions, the resolution policy, and the rationale for each major decision	Wed 30 Sep 2026
D4	Constraint engine module	Source for the standalone engine with its test suite; installable and runnable independently of the interface layer, with a documented programmatic interface	Wed 14 Oct 2026 (M3)
D5	Complete source code	The full application (backend, frontend and infrastructure configuration) in a repository with a layered commit history, and a detailed README	Wed 25 Nov 2026 (M6)
D6	Test suite and coverage report	Unit, integration, adversarial and end-to-end tests; coverage evidencing DS-8; the 40-board regression corpus and 25-brief injection corpus; recorded distributor fixtures for offline replay	Wed 18 Nov 2026 (M5)
D7	Test procedure document	Written procedures for each acceptance test in Table 1: setup, steps, expected result, and how to re-run it, written so the sponsor can execute them independently	Wed 18 Nov 2026 (M5)
D8	Validation evidence pack	Measured results for every specification in Table 1, including usability session results and latency measurements	Wed 18 Nov 2026 (M5)

D9	Deployed system	Live application at a public HTTPS address	Wed 11 Nov 2026
D10	Deployment runbook	Environment variables, secret provisioning, migration procedure, monitoring, rollback, and a documented local-hosting fallback	Wed 25 Nov 2026 (M6)
D11	User guide and training	A written guide covering brief writing, reading a conflict and its evidence, bill-of-materials upload, datasheet attachment and export	Wed 25 Nov 2026 (M6)
D12	Final report and demonstration	Written report covering approach, results against every specification, limitations and recommended future work, plus a 20-minute live demonstration with questions	Wed 25 Nov 2026 (M6)

Table 8: Deliverables and delivery dates

Budget

The budget is dominated by labour. The team estimates the work at 1,106 person-hours in total. Table 9 prices that effort at a notional loaded student-engineer rate of $20.00 per hour and adds the non-labour costs.

Item	Supplier	Quantity	Unit Price	Total
Personnel				
Project Manager — 8 h/wk × 14 wk	-	112h	$20.00	$2,240.00
Lead Developer — 14 h/wk × 14 wk	-	196h	$20.00	$3,920.00
Backend Developer —	-	182h	$20.00	$3,640.00

13 h/wk × 14 wk				
Frontend Developer — 13 h/wk × 14 wk	-	182h	$20.00	$3,640.00
QA Manager, Verification — 11 h/wk × 14 wk	-	154h	$20.00	$3,080.00
QA Manager, Validation — 11 h/wk × 14 wk	-	154h	$20.00	$3,080.00
Release Engineer — 9 h/wk × 14 wk	-	126h	$20.00	$2,520.00
Personnel subtotal		1106h		$22,120.00
Cloud services and data				
Application hosting, 4 months	Render	4 mo	$10.00	$40.00
Managed PostgreSQL, 4 months	Neon	4 mo	$19.00	$76.00
Static site hosting	Render	4 mo	$0.00	$0.00
Language-model inference credits	Z.ai (GLM-5.2)	1	$200.00	$200.00
Component parameters, stock and pricing	JLCPCB	1	$0.00	$0.00

Lifecycle and datasheet data	Mouser	1	$0.00	$0.00
Continuous integration minutes	GitHub Actions	1	$0.00	$0.00
Domain name, 1 year	[Registrar]	1	$15.00	$15.00
Cloud and data subtotal				$331.00
Contingency				
20% on non-personnel items ($331.00)	—	1	$66.20	$66.20
			Total	$22,517.20

Table 9: Requested items and funds for initial design

Communication and Coordination with Sponsor

Communication with the sponsor follows the schedule in Table 10. The Project Manager is the single point of contact and is responsible for every scheduled communication reaching the sponsor on time. The Lead Developer attends the weekly call to answer technical questions and owns any written technical response.

Communication	**Frequency**	**Format**	**From → To**	**Action requested**
Written status report	Bi-Weekly, Thursday	One page: progress against plan, decisions taken, blockers, next week's plan	Mohika → Sponsor, all team members	Information only, unless a blocker is flagged
Sponsor call	Bi-weekly, Thursday	30-minute meeting. Standing	Mohika (chair) and Sparsh	Reply requested on any item

		agenda: status, decisions needing sponsor input, risk review	(technical) → Sponsor	tabled as a decision
Milestone review	At each of M1–M6	45–60 minute demonstration of the exit criterion, with the deliverable circulated 48 hours in advance	Full team → Sponsor	Written sign-off requested to close the phase
Change request	As needed	Written form: what changes, why, schedule and budget impact, and what is removed to accommodate it	Mohika → Sponsor	Written approval required; no scope change is actioned without it
Technical query	As needed	Email or a shared issue-tracker thread	Any team member → Sponsor	Reply requested within 3 working days; the assumption the team will proceed under if no reply arrives is stated in the query
Escalation	As needed	Direct call, followed by written summary within 24	Mohika → Sponsor	Immediate response requested

		hours		
Repository access	Continuous	Sponsor holds read access to the repository and project board from Week 1	—	Information only; the sponsor may observe progress at any time without asking
Final handover	Week 14	Demonstration, training session, and delivery of all artefacts	Full team → Sponsor	Acceptance sign-off requested

Table 10: Communication Plan

Team Qualifications

Mohika brings a strong combination of full-stack software engineering, AI integration, and systems-level problem-solving to the team. She is proficient in Java, Python, and JavaScript with hands-on experience across Spring Boot and React and is well-versed in SQL, and REST APIs. Her internship experience spans building production-grade enterprise systems at CTC Global as well as designing conversational agent architectures for GenAI chatbot. Her strengths in backend engineering, API debugging, cross-layer development, and technical documentation position her to contribute effectively to robust and scalable software solutions.

Faye brings a strong combination of software development, cybersecurity and problem-solving skills to the team. She is proficient in programming languages including Python, Java, C, JavaScript, and SQL with experience developing web applications using Node.js, Vue.js, RESTful APIs, MariaDB and Redis. Her previous projects have involved designing and implementing a real-time machine monitoring website using Agile development practices as well as developing a PyTorch-based deep learning framework. Her experience in system testing, troubleshooting, data analysis and technical documentation further enables her to contribute to developing reliable, secure and well-documented software solutions.

Samarth brings a strong combination of software engineering, test automation, and AI-driven quality assurance to the team. He is proficient in Python, C++, and SQL, with hands-on experience across backend development, REST APIs, microservices, unit and integration testing, and CI/CD. His internship experience spans building and testing enterprise backend systems at SAP as well as developing agentic AI workflows, API simulations, and automated testing solutions using Tricentis Tosca and qTest. His strengths in test automation, API and integration testing, failure analysis, backend

debugging, and AI system evaluation position him to contribute effectively to reliable, well-tested, and production-ready software solutions.

Lai Yi brings a strong combination of frontend software engineering, interactive visualization, and user-interface problem-solving to the team. He is proficient in TypeScript, JavaScript, HTML and CSS, with hands-on experience across React, Vite and Tailwind CSS and is well-versed in REST APIs and server-sent events. His previous projects have involved designing and implementing component-based web applications as well as rendering live backend data as graphs, tables and conflict views. His strengths in typed frontend development, streaming clients, information visualization and usability-focused interface design position him to contribute effectively to a reliable, inspectable and accessible web application.

Sparsh brings a strong combination of agent-system architecture, full-stack engineering, and applied machine learning to the team. He is proficient in Python, SQL (PostgreSQL and MySQL), JavaScript, Java and C, with hands-on experience across React, Django and PyTorch, and is well-versed in LangGraph, the Model Context Protocol, retrieval-augmented generation and vector databases. His internship experience spans building a multi-stage LLM extraction pipeline with a reviewer pass and an LLM-as-judge citation verifier at Dymon Asia Capital, a three-tier validation architecture for a retrieval pipeline at PrepGraph, and production full-stack work at Impress.ai, where he remediated SQL injection vulnerabilities in a PostgreSQL data layer and improved API performance by 20%. His strengths in system architecture, verification and evaluation pipelines, provenance tracking, and technical documentation position him to own the interface contract and the constraint engine, the two artefacts on which the rest of the schedule depends.

Jerral brings a strong combination of AI engineering, backend development, and systems problem-solving to the team. He is proficient in Python, Java, JavaScript, C, and SQL, with hands-on experience across React, Node.js, Express, REST APIs, Docker and CI/CD workflows. During his internship at BRT Systems, he developed AI agents for reviewing Jira requirements, validating bug reports against TestLink cases, and generating unit tests with codebase indexing, reducing manual review effort and improving testing consistency. His previous experience in data center infrastructure and network systems has also given him a strong foundation in troubleshooting and reliability. His strengths in AI agent development, backend engineering, automation, testing, and systems-level problem-solving position him to contribute effectively to building reliable and maintainable software solutions.

Armaan brings a strong combination of AI engineering, machine learning, and full-stack software development to the team. He is proficient in Python, C++, Java, SQL, JavaScript, and TypeScript, with hands-on experience across backend development, REST APIs, React, FastAPI, Docker, and cloud-based data workflows. His internship experience spans building LangGraph-powered agentic AI workflows integrated with Snowflake data pipelines at Circles.Life, developing machine learning solutions for semiconductor defect detection at AMD, and evaluating intelligent document processing systems through structured benchmarking frameworks at RPATech. His project

experience includes developing multi-agent AI systems, event-driven financial backtesting engines, and full-stack web applications using modern frameworks and APIs. His strengths in agentic AI development, machine learning, system integration, data-driven analysis, and scalable software engineering position him to contribute effectively to building reliable, intelligent, and production-ready software solutions.

Conclusion

We propose a system that treats compatibility as the whole circuit board property it actually is, and that it can be trusted when it reports that a board is sound.

Team FUZE proposes to build the proposed system in fourteen weeks. The four objectives are a deterministic constraint engine that validates every rule against every part after every placement, a sourcing pipeline that resolves each slot to a real orderable part while preserving the part's origin, a fenced repair loop in which the model chooses a strategy and the engine re-checks the result, and a streaming interface that makes every judgement checkable by an engineer who did not produce it. Fifteen target specifications convert those objectives into criteria that can be measured on the delivered system, and twelve deliverables give the sponsor something inspectable at every phase gate rather than a report of progress.

References

Accuris and Fuld & Company, The Cost of Inaction: Electronic Parts Intelligence Survey (March 2026). Independent survey of 439 professionals across aerospace and defence, electronics, automotive, medical devices and industrial manufacturing, conducted by Fuld & Company and commissioned by Accuris.
https://accuristech.com/wp-content/uploads/2026/06/The-Cost-of-Inaction-1-2.pdf

Accuris, "The Hidden Tax on Engineering Teams: What Inaction on Parts Intelligence Actually Costs" (7 July 2026).
https://accuristech.com/blog/the-hidden-tax-on-engineering-teams-what-inaction-on-parts-intelligence-actually-costs/

Altium, "Fragmented Feedback Loops: The Hidden Cost in PCB Design and Testing" (28 November 2024). Cited for the contrasting figure of 2.8 respins at $46,000 each.
https://resources.altium.com/p/fragmented-feedback-loops

Anthropic, Model Context Protocol Specification, revision 2026-07-28.https://modelcontextprotocol.io/specification/2026-07-28

CADENAS PARTsolutions, 2022 Engineering Efficiency Report: A Survey among 128,000 Engineers and Designers (2022). https://fs.hubspotusercontent00.net/hubfs/28407/2022 Engineering Efficiency Report.pdf Landing page: https://partsolutions.com/engineering-efficiency-report/

JEDEC, JESD51: Methodology for the Thermal Measurement of Component Packages (Single Semiconductor Device), December 1995. https://www.jedec.org/standards-documents/docs/jesd-51

JLCPCB, Parts Library and PCB Assembly Service. https://jlcpcb.com/parts
LangChain, LangGraph Overview, LangChain documentation. https://docs.langchain.com/oss/python/langgraph/overvie

Lifecycle Insights, 2022 Electronics Design for Resilience (EDfR) Study (2022). Source of the finding that 73 percent of respondents must remove at least one component from each PCB design because of supply, lifecycle or compliance issues. https://www.lifecycleinsights.com/2022-electronics-design-for-resilience-study/ Reported by Siemens at: https://blogs.sw.siemens.com/electronic-systems-design/2022/09/02/can-you-accelerate-the-selection-of-alternate-components/

Mouser Electronics, Search API Documentation, versions 1 and 2. https://api.mouser.com/api/docs/ui/index

Siemens Digital Industries Software, 10 Reasons Growing Teams Choose Xpedition Standard (infographic, 2025). Source of the figures of 2.9 respins per PCB project at an average of $28,500 and sixteen days each, citing Lifecycle Insights research. https://resources.sw.siemens.com/en-US/infographic-10-reasons-growing-teams-choose-xpedition-standard/

Team FUZE, Primary research: fifteen structured interviews with practising hardware engineers and student design teams (2026).

Appendix A: Résumés of Team Members

Mohika Kataruka

Mohika Kataruka | Mobile No.: +65 90845630 | Email: mohika001@e.ntu.edu.sg

EDUCATION

Nanyang Technological University, Singapore Aug 2023 - May 2027
Bachelor of Computing, CGPA: 4.48/5.00

- Honours (Distinction), Current CPA: CGPA: 4.48/5.00 | Minor in Business: Accounting, Fundamentals of Management, Marketing, Management Decision Tools
- Relevant Modules: Data Structures & Algorithms, Software Engineering, Object-Oriented Design & Programming, Artificial Intelligence, Machine Learning, Natural Language Processing

University of Maryland, Maryland, USA Jan 2025 – May 2025
Exchange University

La Martiniere for Girls School, Kolkata, West Bengal, India May 2011 - Apr 2023
Indian School Certificate

- ICSE: 96.80%
- ISC: 96.25%

PROJECTS

ACADEMIC PROJECT
Nanyang Technological University Aug 2024 - Dec 2024
Software Engineering Project - Restaurant Finder

- Engineered a full-stack web application using React and Flask to dynamically filter and rank food establishments based on user-defined dietary constraints and location parameters.
- Integrated the Google Places API to ingest and structure dynamic location metadata, maintaining search relevance and query response efficiency.

INTERNSHIP EXPERIENCE

TATA Group June 2026 - July 2026
GenAI Chatbot Enhancement and Implementation Intern

- Co-designed production-grade conversational agent architectures deployed across 500+ digital properties; established the system's intent taxonomy, dialogue state-machine management, and API integration paths.
- Owned end-to-end UAT and QA cycles, developing 500+ automated and manual conversation-flow test cases to proactively trace, isolate, and debug edge-case dialogue failures pre-deployment.
- Audited production platforms across Taj Hotels' core digital footprint, developing a zero-migration service-layer abstraction for cross-property content deduplication.

CTC Global Pte Ltd, Singapore May 2025 - Dec 2025
Software Engineering Intern

- Architected an enterprise-level Timesheet Management System with secure role-based access control (RBAC) and automated reporting engines for 300+ corporate users.
- Refactored Contract Management (CSMS) REST APIs by implementing server-side caching, filtering, and custom page-indexing algorithms, reducing API response times by roughly 30%
- Engineered transaction-safe data access mechanisms across Spring Boot Controller, Service, and DAO layers, eliminating SQL projection issues and reducing null-pointer exceptions by 80%
- Refactored a legacy Dropbox Automation Tool, resolving 40+ SonarQube-identified maintenance and security warnings to secure client-side production sign-off.

LEADERSHIP EXPERIENCE

Nanyang Technological University Student Union Aug 2025 – Present
Student Integration Committee - Chairperson

- Lead a cross-functional student team comprising 100+ members, managing resource allocation and strategic program alignments across 7 university student clubs.
- Manages budgets and end-to-end operational execution for high-turnout campus initiatives, coordinating safety, logistics, and scheduling for up to 1,000+ participants.
- Preserved baseline participation rates of [X]+ annual attendees despite a SGD 6,000 budget reduction by restructuring procurement and vendor logistics.

SKILLS

- **Programming:** Java, Python, JavaScript, HTML/CSS
- **Frameworks & Tools:** Spring Boot, Flutter, React, Flask, FastAPI, Git, OutSystems
- **Databases & APIs:** SQL, REST APIs, Postman
- **Languages:** English (Fluent), Hindi (Fluent)

Faye Chan

Faye Chan | +65 8488 5361 | fayechan125@yahoo.com | linkedin.com/in/fayechan | github.com/FayzeC

EDUCATION

Nanyang Technological University, Singapore — Aug 2023 – Present
Bachelor of Computing (Computer Science with Specialisation in Security)
- Current CGPA: 4.37/5.00
- Expected Degree Completion: Dec 2026

Singapore Polytechnic, Singapore — Apr 2020 – Apr 2023
Diploma in Infocomm Security Management
- Dean's List for Academic Year 2020/2021

EXPERIENCE

Synapxe Pte Ltd — Jul 2025 – Dec 2025
Cyber Risk Management and Data Security Risk Assessment Intern
- Automated an Excel VBA Residual Risk Assessment template with dynamic validation, conditional fields and risk-based controls, reducing manual errors and helping complete project Phases 1 and 2 ahead of schedule
- Consolidated and analysed sector-wide risk assessments to identify high-risk systems, compliance trends and security control gaps, supporting risk prioritisation and remediation planning across public healthcare clusters
- Guided technology teams and external stakeholders through the assessment process by resolving submission issues and translating technical controls, policy requirements and compliance criteria into actionable guidance

Thinkture Pte Ltd — Sep 2022 – Feb 2023
Associate Security Consultant Intern
- Executed System Security Acceptance Tests on switches, routers, PostgreSQL databases and docking stations
- Performed system audits, firewall rule planning, and deployed, maintained and troubleshot a Fortinet FortiGate next-generation firewall and Symantec Endpoint Protection Manager (SEPM) to strengthen client network security
- Produced 3 installation guides and technical research documents to support product deployment

CYBERSECURITY ACHIEVEMENTS & CERTIFICATIONS

- DSO Capture the Flag Challenge 2026 – Cash Prize Awardee (Cleared both Easy & Hard challenge categories)
- Google Cybersecurity Professional Certificate – Google (Coursera), 2026

ACADEMIC PROJECTS

Nanyang Technological University, Singapore
Final Year Project – Deep Learning Watermarking Using Network Trojan Techniques — Jan 2026 – Present
- Developed a PyTorch-based deep learning watermarking framework using patch-based, warping-based and input-aware dynamic triggers for ownership verification on CIFAR-10 ResNet-18 models
- Standardised evaluation and assessed watermark effectiveness, functionality preservation and robustness against fine-tuning, pruning, transfer learning and input transformation attacks

Singapore Polytechnic, Singapore — Apr 2022 – Aug 2022
Final Year Project – Machine Monitoring Website (Industry Collaboration with Leaptron Engineering)
- Designed and developed a real-time equipment monitoring website using Node.js, MariaDB, Redis and Vue.js, enabling clients to log in, add machines to production lines and track Overall Equipment Effectiveness (OEE)
- Implemented Agile methodology, increasing project efficiency by 20% through sprint-based development, ensuring iterative improvements and continuous alignment with client requirements

SKILLS

Programming & Scripting: Python, C, Java, JavaScript, VBA, SQL, HTML, CSS
Cybersecurity & Risk: Cyber Risk Assessment, Data Security Risk Assessment, Network Security, System Security Acceptance Testing, Security Product Deployment and Troubleshooting, Firewall Rule Planning, Reverse Engineering
Machine Learning: PyTorch, Deep Learning, ResNet-18, Model Robustness Evaluation
Data Analysis & Documentation: Data Cleaning and Visualisation, Compliance Reporting, Technical Documentation
Web & Databases: Node.js, Vue.js, RESTful APIs, MySQL, MariaDB, Redis
Systems & Tools: Linux, VMware Workstation, Ghidra, WinDbg, Wireshark, Microsoft Excel
Languages: Proficient in English, Mandarin Chinese, Malay

Samarth Jethani

SAMARTH JETHANI

+65 85900799 | samarthjethani@gmail.com |GitHub |LinkedIn

EDUCATION

Nanyang Technological University (NTU), Singapore **Expected 2027**
Bachelor of Computing (Artificial Intelligence), Minor in Mathematics & Business | GPA: 4.40/5.00 (Distinction)
Exchange: University of Maryland, College Park (Spring 2026) | Coursework: Machine Learning, NLP, Databases, Linear Optimization, Advanced Computer Architecture, Linear Algebra

EXPERIENCE

Tricentis - Solution Architecture Intern (Forward Deployed Engineering) **Jun 2026 - Aug 2026**
- Led client-facing POC demonstrations of Tricentis AI Workspace for enterprise prospects across APAC, owning demo narrative, solution positioning and live technical Q&A.
- Built ~20 agentic AI workflows, including a 13-stage requirements-to-automation pipeline across Confluence, qTest and Tosca with human approval gates before automation writes.
- Engineered a stateful Tosca Cloud API Simulation POC virtualising a full checkout journey across authentication, catalog, inventory, payment and shipping, with fault injection and stateful responses.
- Beta-tested AIDA, Tricentis' agentic AI testing product; authored the TriServe demo project spanning qTest, Tosca and Salesforce.

SAP Asia - AI & Backend Development Intern (Software Engineering) **Jun 2025 - Dec 2025**
- Engineered and hardened enterprise backend services for quota and access-control workflows, applying production software engineering practices around correctness, reliability and failure-safe behavior.
- Implemented mTLS authentication and authorization for service-to-service APIs and built automated unit and integration tests protecting security-critical behavior against regressions.
- Deployed secure backend services on SAP BTP within a multi-tenant architecture using IAS/XSUAA authentication and enterprise access-control policies.
- Worked across backend, security and deployment concerns in an enterprise codebase, strengthening hands-on experience with API design, testing, cloud services and production debugging.

Get My Parking - AI & Automation Intern **May 2024 - Aug 2024**
- Collaborated across engineering and business teams to develop an OCR + LLM pipeline processing 3,000+ heterogeneous parking-rate documents, translating an operational data problem into an automated technical workflow.
- Improved structured extraction accuracy from ~78% to ~91% through prompt refinement and rule-based validation, then automated downstream pricing workflows to reduce manual review effort by ~40%.

Kinetiz Racing Team (A2RL) - Localization Researcher **Feb 2024 - Feb 2025**
- Developed real-time sensor-fusion localization algorithms combining IMU and vision data for a full-scale autonomous Formula race car.
- Contributed to path-planning and decision-making systems deployed on the autonomous racing platform, working at the intersection of robotics, perception and real-time systems.

SELECTED PROJECTS

Enterprise AI Operations Copilot **Python | FastAPI | LLMs | RAG | PostgreSQL | Docker**
- Developed an enterprise AI application translating natural-language business requests into multi-step workflows across documents, structured data and external APIs.
- Implemented tool-using agents, retrieval, human-in-the-loop approvals, audit trails and failure recovery; designed modular integrations for onboarding new business processes and data sources.

AI Decision Intelligence Platform **Python | SQL | FastAPI | PostgreSQL | React | Docker**
- Developed an end-to-end analytics platform translating business questions into validated queries, executing analysis across enterprise datasets and producing decision-ready explanations and visualizations.
- Implemented schema-aware query generation, automated validation, role-based access controls and reusable data connectors to adapt the platform across datasets and business domains.

AI Deployment & Evaluation Platform **Python | FastAPI | Docker | CI/CD | LLM APIs**
- Developed reusable infrastructure to configure, evaluate and deploy AI applications across multiple models, prompts and business use cases through production-oriented services.
- Built automated evaluation for quality, latency, cost and failure modes alongside observability, versioned experiments, reproducible deployment and rollback workflows.

TECHNICAL SKILLS

Languages: Python, C++, SQL
AI / ML: LLMs, Agentic AI, RAG, Machine Learning, NLP, Prompt Engineering, Model Evaluation, OCR
Software Engineering: Backend Development, REST APIs, Microservices, System Design, API Integration, Unit & Integration Testing, mTLS
Cloud / Platforms: SAP BTP, AWS, Docker, CI/CD, Git, Confluence, qTest, Tosca, Salesforce
Certifications: AWS Certified Cloud Practitioner | DeepLearning.AI Machine Learning Specialization

Armaan Sinha

ARMAAN SINHA

Mobile No.: +65 9885 2472 | Email: armaansinha05@gmail.com | LinkedIn | GitHub

EDUCATION

Nanyang Technological University, Singapore Aug 2023 – May 2027
COLLEGE OF COMPUTING AND DATA SCIENCE (CCDS)
Bachelor of Computing, Computer Science, Minor in Mathematics

- 4.17/5
- Exchange Student – University of Waterloo, Waterloo, Ontario, Canada (Spring 2026)
- Relevant Modules – Probability, Ordinary Differential Equations, Software Engineering, Linear Algebra, Discrete Mathematics, Financial Markets, Algorithm Design and Analysis, Convex Optimization, Calculus-III

AWARDS AND HONOURS

- Indian Institute of Technology Joint Entrance Examinations (IITJEE): Ranked in **top 3%** in JEE Main and **top 10%** in the JEE Advanced, with a high score in Physics, 2023.

EXPERIENCE

Circles.Life, Singapore May 2026 – Aug 2026
AI Engineering Intern (CEO Office)

- Engineered a LangGraph-powered agentic AI workflow with Snowflake data pipeline integrations (RSA auth, Snowflake tasks, automated refresh, SQL API), automating common customer support resolutions for production deployment targeting 100,000-500,000+ users.
- Authored a research report on AI integration strategy, analyzing 7+ major telco case studies across 4 global regions to surface actionable recommendations for improving customer satisfaction.

Advanced Micro Devices (AMD), Singapore Jun 2025 – Dec 2025
Machine Learning and SWE Internship

- Developed multiple Machine Learning solutions and generated 100+ labeled samples for defect detection in microprocessors, reducing production time by approximately 30-40%.
- Assisted the team by providing automation solutions with Power BI, LLM platforms, and Python scripts that streamlined daily processes and increased team efficiency by 30%.

RPATech May 2024 – Jul 2024
AI Research Internship

- Evaluated 7 Intelligent Document Processing (IDP) tools (4 premium, 3 open-source) by developing a standardized assessment framework to benchmark cost, efficiency, accuracy, adaptability, and ease of use, analyzing strengths and limitations and documenting the findings in an internal research paper.

PROJECTS

ATLAS

- 5th/400+ at BUILD2026 by 'SUP – Multi-agent Student OS on FastAPI and React/Vite that auto-triages academic deadlines and coursework by orchestrating semantic search (Exa API), email agent, GPT, TTS and an animated UI.

BACKTESTER

- Built a Python event-driven backtesting engine using pandas, NumPy, yfinance, OOP, and CLI tooling, implementing SMA crossover and RSI mean-reversion strategies with slippage, commissions, portfolio tracking, walk-forward validation, and Sharpe/Sortino, CAGR, beta, and drawdown analysis.

SCHOOL4U

- Created a Singapore-based school discovery platform using React, Node, Express and Supabase, with location-based search, JWT authentication, real-time messaging, and REST API integration.

LEADERSHIP EXPERIENCE | CO-CURRICULAR ACTIVITIES

NTU Student Union – Chairperson Sep 2024 – Aug 2025

- Led a diverse 14-member team to plan and execute welfare programs and events impacting 1000+ NTU students annually by collaborating with internal and 5+ external partners to enhance student well-being.

NTU International Chess Club – Captain Aug 2023 – Aug 2024

- Achieved FIDE Elo 1668 (ID: 25674102) and captained NTU Varsity Team B.

SKILLS

- **Programming Languages:** Python, C++, Java, SQL, JavaScript, TypeScript, HTML, CSS3, Go.
- **AI/ML & Data Science:** TensorFlow, PyTorch, NumPy, Pandas, Matplotlib, Neural Networks, Computer Vision, Natural Language Processing, Transformers, OCR, IDP, Statistical Analysis, Regression Analysis.
- **LLM/Agentic AI:** LangChain, LangGraph, Agentic AI Workflows.
- **Frameworks, Tools & Platforms:** Docker, Git, Snowflake, Version Control, Bitbucket, Prometheus, Power BI, Firebase, Supabase, React, FastAPI (Full-Stack Web Development Proficiency).
- **Languages:** English, Hindi - All professional proficiency or above.

Jerral Chua Fung Yong

JERRAL CHUA FUNG YONG | Singaporean | Mobile No.: +65 9324 2599

Email: jcfy123@gmail.com | GitHub | LinkedIn

EDUCATION

Nanyang Technological University (NTU), Singapore Aug 2023 - May 2027 (expected)
Bachelor of Computing (Hons) in Computing Science
Honors: Distinction, CGPA: 4.3/5.0
- Overseas Exchange: Luleå tekniska universitet
- Related Modules: Machine Learning, Natural Language Processing, Artificial Intelligence, Computer Vision

NGEE ANN POLYTECHNIC (NP), Singapore Apr 2018 - Apr 2021
Diploma in Network System and Security
CGPA: 3.75/4.0
- Singtel Cadet Scholarship

SKILLS & CERTIFICATIONS

- Languages: Python, Java, JavaScript, C, SQL
- Web & Backend: React.js, Node.js, Express, HTML/CSS, REST APIs, MQTT/WebSocket
- Tools & Practices: Git, Docker, Jenkins (CI/CD), Jira, Agile/Scrum, Unit Testing (JUnit, pytest), OOP, Design Patterns
- AI & Data: LLM agent development, prompt engineering, Pandas, scikit-learn
- Certifications: Google IT Support Fundamentals; Google Bits and Bytes of Computer Networking (2021)
- Languages (Spoken): English, Mandarin Chinese

WORK EXPERIENCE

BRT Systems Pte Ltd, Software Department, AI Engineering Intern Jan 2026 - Jul 2026
- Built a Python agent running on Jenkins that reviews Jira Stories for requirement quality posts its findings as comments and rewrites weak descriptions into ADF format. Handles 60 stories a month and cuts manual review time by 50%.
- Created an agent that checks bug reports for completeness, cross-references them against linked TestLink test cases, and flags gaps for the team. Covered 30 reports a month across all projects.
- Wrote a unit test generation agent with codebase indexing for Python and TypeScript, holding coverage variance under 3% between runs.

Singapore Armed Forces (National Service), Armament Technician Specialist Nov 2021 - Aug 2023
- Led a team of 10 technicians maintaining 200+ battalion vehicles and equipment units, sustaining a 95% operational readiness rate and 98% of tasks completed on or ahead of schedule.
- Built a tracking system for 100+ spare parts that cut shortages by 40% and kept mission-critical components available.

SingTel, Data Center Infrastructure Management Intern Apr 2020 - Sep 2020
- Supported operations across all data centers, introducing proactive monitoring that raised uptime to 99.9% and deploying 250+ temperature sensors to fix cooling inefficiencies and cut energy use.
- Contributed to an internal web tool for service requests that shortened response times by 20% for the operations team.

Projects

Lulea Formula Student (LFS) VCU Telemetry Dashboard | JavaScript, Node.js, Express, MQTT, WebSocket
- Built a live telemetry dashboard for a Formula Student team's vehicle control unit, streaming car data over MQTT to the browser via WebSocket so the page updates in real time with no backend relay.
- Structured the front end as a shared base module plus per-page modules so the connection and UI logic sit in one place, and rendered the gauges with the Canvas API rather than pulling in a chart library.

Rivals for Catan, Refactored Game Engine | Java, Maven, JUnit 5, Gson
- Refactored a monolithic Java codebase into 22+ classes across 5 packages, applying SOLID principles so new cards or player types can be added without changing the engine.
- Applied four design patterns. Strategy for card effects, factory for card creation, observer for game events, and template method for turn flow. Added a proxy so remote socket players use the same interface as local ones.
- Wrote JUnit 5 tests for card loading, dice production, event handling, and win conditions, with all 94 cards loaded from JSON so game data lives outside the code

ParkIT, Parking Management Web App | JavaScript, React.js, Node.js, Express, MongoDB, JWT
- Built the backend for a full-stack parking app in a team of 6, exposing REST APIs for real-time car park availability, JWT-based auth and check-in/check-out sessions, integrating with Singapore's HDB parking API.
- Designed the MongoDB data models and endpoints behind expense tracking, favorite carparks, and the feedback system, validated with Express Validator.

LEADERSHIP & CO-CURRICULAR ACTIVITIES

NTU Crescent Hall Transition Orientation Programme, Business Manager Sep 2023 - Aug 2024
- Ran three fundraising events and secured sponsorships from 10+ organizations, raising over $1,000 in cash plus in-kind resources for the program.
- Managed the welfare package workstream end to end, delivering 150+ customized packs to incoming students.

NTU Crescent Hall Softball Club, Captain Sep 2023 - May 2024
- Led regular practices focusing on teaching rules and skills, leading to the team being awarded participation in interhall.
- Liaised with the hall council to procure necessary equipment and team jerseys.

Sparsh Jain

SPARSH JAIN | +65 81913177 | sparsh002@e.ntu.edu.sg | github.com/sparsh-555 | linkedin.com/in/sparshja1n

EDUCATION

Nanyang Technological University, Singapore — Jul 2023 – Jul 2027
Bachelor of Computing in Computer Science with a Second Major in Business

University of Florida, FL — Aug 2025 – Dec 2025
Exchange Semester, Computer Science

WORK EXPERIENCE

Dymon Asia Capital — Jan 2026 – Present
AI Research Intern
- Built a multi-stage LLM pipeline extracting structured trade ideas from sell-side research across 14+ banks (FX, rates, options), with a reviewer pass and nightly reconcile for correctness
- Shipped the firm's first stateful deep agent, migrating the analyst assistant from single-turn to multi-step research with cross-session memory, MCP tool calls, and file generation on Anthropic Managed Agents
- Prototyped an autonomous trade-research agent with a generator/evaluator/critic loop that blind-backtests each idea over 3 years of daily bars through TradingView MCP and writes lessons to memory
- Developed an LLM-as-judge citation-verifier that validates each cited claim against its source chunk before users see it, and fixed a ~14× collapse in retrieval token throughput from a token-vs-character chunking bug and non-ASCII stripping that was dropping CJK research

PrepGraph (Part-Time, Remote) — Sep 2025 – Nov 2025
AI Engineer Intern
- Engineered a three-tier validation architecture for a RAG question-generation pipeline, reaching 85% success on advanced fingerprints and 100% on basic validation from a 0% baseline
- Cut question-generation API latency 60% (10s to 4s) by moving quality checks to async workers
- Reduced LLM hallucinations to <1% with a context-aware summarisation layer and prompt redesign

Impress.ai — May 2025 – Aug 2025
Full Stack Developer Intern
- Shipped a production WhatsApp chat interface (React, Django) for real-time recruiter-candidate messaging
- Found and remediated SQL injection vulnerabilities in the PostgreSQL data layer
- Optimised RESTful APIs and database queries, improving platform performance 20%

Motherson Group — May 2024 – Aug 2024
Cloud Infrastructure & AI Engineer Intern
- Built an AI supplier-discovery chatbot on Oracle Cloud (Llama 3 8B) with a weighted multi-criteria ranking algorithm over pricing, quality and delivery, improving supplier-match accuracy ~35%

PROJECTS

Nexa: Systematic Trading & Portfolio Optimization — Aug 2025 – Dec 2025
ML Course Final Project
- Engineered a hybrid deep learning architecture combining Temporal Fusion Transformers (TFT) and GRUs to generate multi-horizon (T+14) stock price forecasts
- Implemented a Reinforcement Learning agent (PPO/DDPG) to dynamically optimize portfolio allocation, maximizing the Sharpe Ratio while minimizing drawdown
- Developed an intelligent feature engineering pipeline using ensemble clustering and News API sentiment analysis to extract 80+ technical and alpha indicators
- Integrated a Self-Reorganizing Fuzzy Logic (SeroFAM) layer to provide explainable trading signals, reducing model opacity compared to traditional black-box architectures

Hierarchical Multi-Agent RAG System — Aug 2024 – May 2025
Undergraduate Research Experience on Campus @ NTU
Student Researcher
- Designed and implemented a three-agent RAG architecture grounded in Zimmerman's Self-Regulated Learning theory, using fine-tuned GPT-4o classifier to automatically route programming queries to specialized debugging or implementation agents based on student learning phase
- Fine-tuned GPT-4o on labeled CS1QA dataset using 70/30 train-test split to distinguish between performance phase (debugging) and planning phase (implementation) queries, achieving 91.4% routing accuracy on held-out test set through context-aware classification

Lai Yi

LAI YI

cradlelinerstar@gmail.com | linkedin.com/in/Lai-Yi-Larry | github.com/CradleStrife

EDUCATION

Nanyang Technological University, Singapore Aug 2023 – May 2027 (expected)
Bachelor of Computing in Computing and Data Science

- CGPA: 4.47 / 5.0
- Relevant modules: Software Engineering, Data Structures & Algorithms, Artificial Intelligence

The University of Tokyo, Japan Jun 2024 – Sep 2024

- GUC Summer Research Programme (Group Theory and Artificial Intelligence)

SKILLS & CERTIFICATIONS

- **Languages:** TypeScript, JavaScript, Python, Java, C, HTML/CSS, SQL
- **Web:** React.js, Vue.js, Vite, Tailwind CSS, Firebase, REST APIs, Django, Flask
- **AI / ML:** PyTorch, Transformers, Hugging Face, ComfyUI
- **Tools:** Git, Docker, AWS, Elasticsearch, Linux, Axure
- **Certifications:** Google IT Support; Google Advanced Data Analytics
- **Spoken:** English, Mandarin Chinese

EXPERIENCE

iFLYTEK, Shanghai Dec 2025 – May 2026
AI Algorithm Intern — LLM-based Speech Synthesis

- Documented an internal LLM-based TTS pipeline end to end, from data preprocessing to acoustic synthesis.
- Ran multilingual TTS data jobs on a GPU cluster, covering feature packing, alignment and quality checks.
- Automated preference-pair annotation with forced alignment and auxiliary ASR; evaluated on 280 test cases.

Alibaba Group, Hangzhou Jun 2025 – Sep 2025
AI Intern & Product Manager

- Built a prompt-driven semantic cut-out workflow (Florence2 + SAM2) for object segmentation and background removal in ComfyUI.
- Trained a multimodal generative model for personalized product imagery and integrated it into an internal generation workflow.

Singapore Food Agency Nov 2024 – Mar 2025
Full-Stack Intern

- Built interactive dashboards and frontend views for an internal analytics platform in Vue.js.
- Delivered full-stack services with Django, Docker, Elasticsearch and AWS, exposing REST APIs consumed by the UI.

Chinese Academy of Sciences Dec 2024 – Feb 2025
Software Engineering Intern

- Implemented an early web research platform combining interactive data views with database-backed workflows.
- Prototyped the interface in Axure 11 and restructured frontend and backend modules for clearer separation of concerns.

PROJECTS

SpeechViz | TypeScript, JavaScript, Web Speech API, MobX Sep 2024 – Apr 2025
NTU Multimedia and Interactive Computing Lab

- Built the client pipeline from speech recognition to LLM intent parsing to chart operations (select, filter, zoom, aggregate).
- Implemented silence detection, domain grammar rules and MobX state management for multi-turn voice interaction.

MediMap+ | React.js, Firebase, Google Maps API Course project
Deputy team lead — frontend

- Built most of the frontend: map-based search, accounts, appointments and notification flows.
- Integrated a third-party email service and wrote part of the technical documentation.

LEADERSHIP

NTU Chinese Society — Publicity Committee Sep 2024 – Present

- Directed event publicity and visual materials; previously deputy team lead for MediMap+ frontend.